

Data Structure Project 1

Ye Jinhao

24300750085@m.fudan.edu.cn

September 27, 2025

1 Abstract

The project first implemented several sorting algorithms, including insertion sort, merge sort, quick sort, and a hybrid sort combining insertion sort and quick sort. Then I explore the performance of each sorting algorithm, finding that the quick sort transcends the insertion sort when $N > 35$. Finally, I derived that the theoretical best k for the hybrid sort is strictly related to the point N that quick sort transcends insertion sort, irrelevant with the size of arrays. I also did several experiments to obtain the best k in my machine, which is about 15 19.

2 Problem 1,2,3,5

See in the code submitted.

3 Problem 4

Problem 4 asked me to find the optimal k for the hybrid algorithm combining insertion sort and quicksort. I'm going to argue that the optimal k is related to the cross point of insertion sort and quicksort N , with the relation of $k = \frac{N}{2 \ln N}$.

3.1 Theoretical Derivation

3.1.1 Time Complexity of Insertion Sort

Insertion sort works by building a sorted subarray one element at a time. For each element, I compare it backward with the already sorted part until I find its position.

- In the **best case**, the input array is already sorted. Each insertion only requires one comparison, so:

$$T_{\text{best}}(n) = \Theta(n).$$

- In the **worst case**, the input array is in reverse order. The i -th element requires i comparisons and moves:

$$T_{\text{worst}}(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2).$$

- In the **average case**, about half of the elements in the sorted part are compared for each insertion:

$$T_{\text{avg}}(n) = \Theta(n^2).$$

3.1.2 Time Complexity of Quicksort

Quicksort uses the divide-and-conquer paradigm. It partitions the array around a pivot, then recursively sorts the two subarrays.

- The **partition** step takes $\Theta(n)$ time.
- In the **best case**, the pivot splits the array into two equal halves each time:

$$T_{\text{best}}(n) = 2T\left(\frac{n}{2}\right) + \Theta(n).$$

Applying the Master Theorem:

$$T_{\text{best}}(n) = \Theta(n \log n).$$

- In the **worst case**, the pivot always produces one empty subarray and one subarray of size $n - 1$:

$$T_{\text{worst}}(n) = T(n-1) + \Theta(n) = \Theta(n^2).$$

- In the **average case**, the pivot tends to produce reasonably balanced splits. The recurrence is:

$$T_{\text{avg}}(n) = 2T\left(\frac{n}{2}\right) + \Theta(n),$$

leading to:

$$T_{\text{avg}}(n) = \Theta(n \log n).$$

or:

$$T(n) = cn \log n$$

3.1.3 Analysis of Hybrid Sort

Now we are going to dive into the time complexity of hybrid sort. Assuming that the quick sort still works in the average case, splitting the array to almost balanced parts, then the running time can be recursively expressed as:

$$T(n) = \begin{cases} c_1 n^2, & n \leq k, \\ 2T\left(\frac{n}{2}\right) + c_2 n, & n > k, \end{cases}$$

In which c_1, c_2 represents the running time constants. Solving the expression we get:

$$T(n) = c_2 n \log_2 \frac{n}{k} + c_1 n k$$

In order to optimize k mathematically, we treat k as a continuous variable. Derivative with respect to k , and we have:

$$\frac{dT}{dk} = n \left(-\frac{c_2}{\ln 2} \cdot \frac{1}{k} + c_1 \right).$$

Let it = 0, we have:

$$k^* = \frac{c_2}{c_1 \ln 2}$$

However, the expression still contains the unknown constants. Luckily, we can express c_1/c_2 by the cross point N . Here we try to derive the N :

We model insertion sort and (average-case) quicksort by

$$T_{\text{ins}}(n) = c_1 n^2, \quad T_{\text{qs}}(n) = c_2 n \log_2 n,$$

where $c_1, c_2 > 0$ absorb constant factors. Let N be the empirical *crossover size* such that $T_{\text{ins}}(N) = T_{\text{qs}}(N)$. Then

$$c_1 N^2 = c_2 N \log_2 N \implies \frac{c_2}{c_1} = \frac{N}{\log_2 N}. \quad (1)$$

For the hybrid algorithm that uses insertion sort when subproblem size $\leq k$ and otherwise recurses with quicksort, the exact (aligned) cost is

$$T(n) = c_2 n \log_2 \frac{n}{k} + c_1 n k,$$

whose minimizer (treating k as a positive real) is

$$k^* = \frac{c_2}{c_1 \ln 2}. \quad (2)$$

Substituting (1) into (2) gives

$$k^* = \frac{1}{\ln 2} \cdot \frac{c_2}{c_1} = \boxed{\frac{N}{\ln N}}.$$

3.2 Experiment Validation

In order to verify the theoretical result of k and find the true k on my machine, I implemented three experiments.

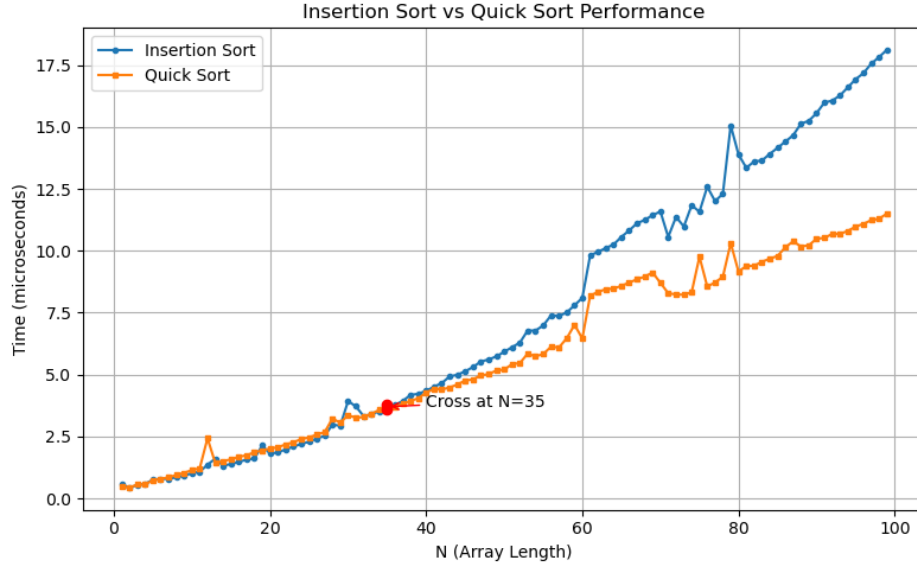


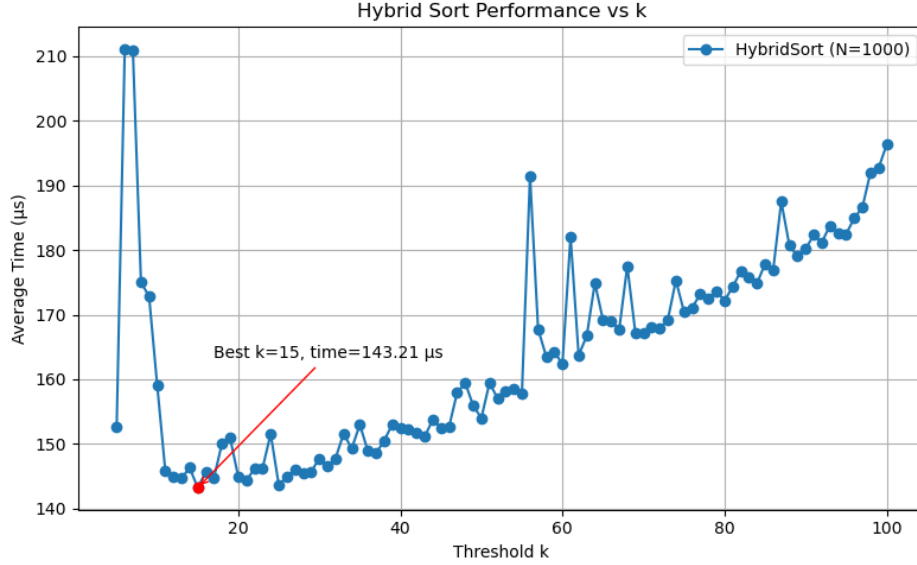
Figure 1: The Running Time of Insertion Sort and Quicksort

3.2.1 Experiment 1: Finding the Cross Point N

We tested the running time of insertion sort and quicksort by repeating to adapt them on randomized arrays of size from 1 to 100. Results see in `result1.csv`. Here shows that the cross point is $N = 35$.

3.2.2 Experiment 2: Test the Optimal k On My Machine

In order to optimize the k, we fixed the length of the array at 1000, and tested k range from 5 to 100. For each k, 1000 randomized arrays were tested and get an average running time.



Note: Array length N is fixed at 1000

Figure 2: The Optimal k

By experiment, we find that the optimal k in my machine is about 15. By substituting $N = 35$ into the theoretical formula $k = \frac{N}{\ln N}$, we obtain that the theoretical value of k should be about 9.8. The mismatch of experiment and theory indicates the complexity and biases in our machine environment.

3.2.3 Experiment 3: Verify that k is Independent of the Array Length.

Due to our theoretical derivation, we found that k is independent to the length of array. To verify the conclusion, we design a experiment to include array size as a variable, ranging in $\{50, 100, 500, 1000, 5000, 10000\}$. We found that the optimal k ranges from 15 to 20, indeed almost unrelated to the length of the array.

4 Conclusion

In this project, I have explored the performance of insertion sort, quicksort, and a hybrid algorithm that combines the two. By both theoretical analysis and experimental validation, I derived the closed form of the hybrid algorithm's running time and proved that the optimal threshold k can be expressed by the crossover point N between insertion sort and quicksort, with the relation $k \approx N / \ln N$. This result shows that the choice of k is independent of the input array length and is instead determined by the relative efficiency of the two underlying algorithms.

Experiments confirmed the theoretical framework but also revealed interesting deviations. On my machine, the crossover point was observed at $N = 35$, which gives a theoretical threshold of $k \approx 9.8$. In practice, however, the optimal k was consistently measured around 15–19. The discrepancy arises because the theoretical model only counts asymptotic

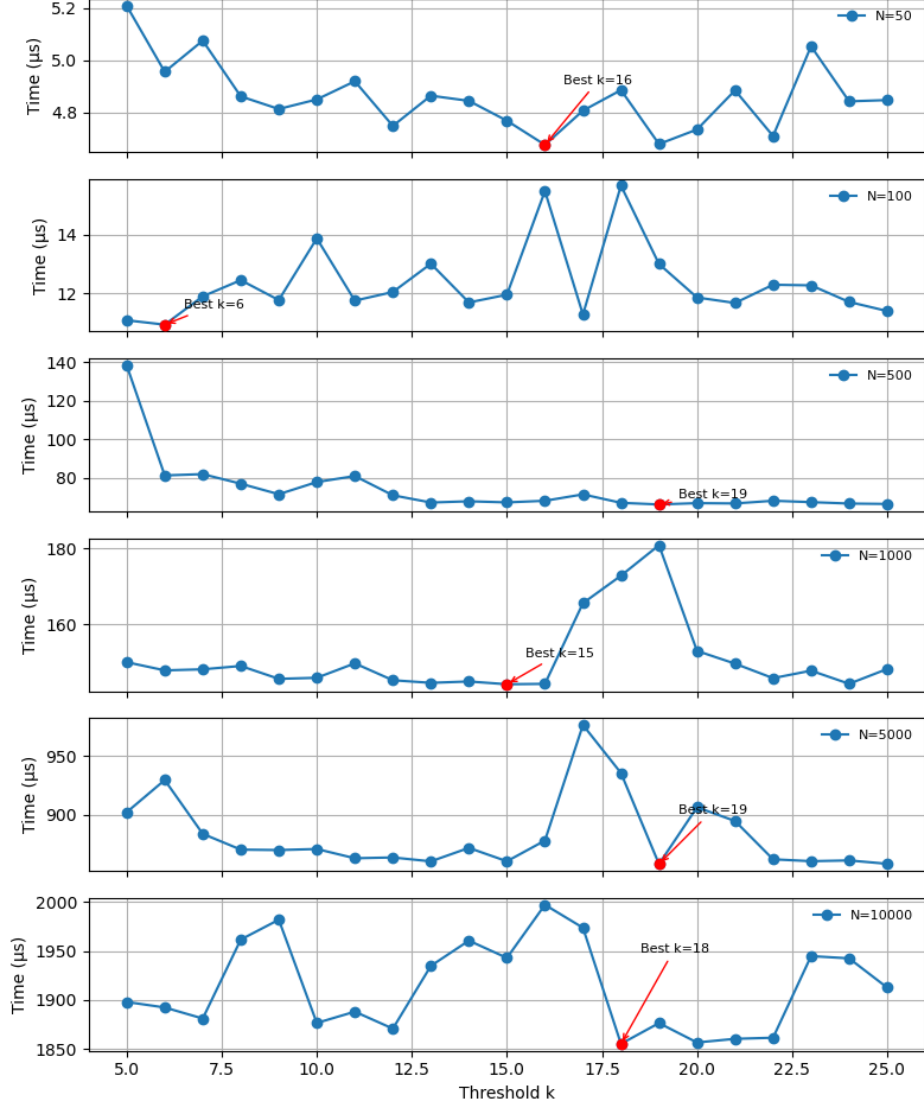


Figure 3: k is unrelated to the length of arrays

operations, while real-world execution is influenced by factors such as constant overheads, cache performance, compiler optimizations, and even random fluctuations in system scheduling. These effects enlarge the practical benefit of insertion sort on small subarrays, thereby pushing the empirical k to a higher range than the pure formula predicts.

Overall, the study demonstrates both the power and the limitations of theoretical analysis. The hybrid strategy achieves the expected efficiency gain, while careful experiments are still necessary to fine-tune the optimal parameters on a specific computing environment.